

Complessità Computazionale

Tempo e Spazio, Andamento Asintotico, Notazione O e Θ ,
Caso migliore e peggiore

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio

Tempo e Spazio di Calcolo

Complessità Computazionale: Il Problema

- In precedenza abbiamo visto com'è possibile migliorare l'efficienza di un programma C
 - Cambiando alcuni dettagli nell'implementazione delle strutture dati e delle loro operazioni
 - Ad es.: le diverse implementazioni per il tipo di dato "numero complesso"
- La **complessità computazionale** descrive come variano
 - il **numero** complessivo dei **passi** (o istruzioni) elementari eseguiti
 - la **quantità** di **memoria** utilizzatadurante l'esecuzione di un programma
 - al crescere della **dimensione** dei **dati** in input
- Tenete presente che:
 - Le nozioni presentate sono indipendenti dal particolare linguaggio di programmazione
 - Il linguaggio C può essere visto come una notazione per scrivere algoritmi
 - Una notazione eseguibile (modulo traduzione) da una macchina fisica (cioè, un calcolatore digitale)

Passi Elementari

- Il tempo necessario per eseguire
 1. un dato programma
 2. con un dato inputdipende dalla macchina sulla quale si esegue il programma
- Le macchine fisiche (cioè, i calcolatori digitali) che eseguono i programmi C funzionano eseguendo una serie di passi (o istruzioni) elementari
 - I passi si possono considerare tutti della stessa durata (ad es. la durata del passo più lungo)
- Quindi:
 - Il tempo di calcolo è proporzionale al numero di passi elementari eseguiti
 - Possiamo limitarci a contare i passi elementari

Ma cosa sono e come si contano i passi elementari?



Passi Elementari: Esempi



- Possiamo assumere che un passo elementare sia l'esecuzione di una delle seguenti operazioni o istruzioni di un programma C¹:
 - le operazioni
 - aritmetiche (+, -, *, /, %, ...)
 - confronto (<, >, ==, <=, ...)
 - operazioni logiche (&&, ||, !, ...)
 - le operazioni di lettura o scrittura di una cella di memoria
 - accesso al valore contenuto in una variabile
 - assegnazione di un valore ad una variabile
 - accesso ad un elemento di un array attraverso l'indice
 - accesso ad un membro di una struct
 - ecc.
- le istruzioni di controllo
 - salto al ramo then o else di un if in base al risultato del test
 - salto all'inizio del corpo di un ciclo o al uscita dal ciclo in base al risultato del test
 - salto al test di un ciclo al termine dell'esecuzione del corpo
 - chiamata di funzione
 - ritorno da funzione
 - ecc.
- ecc.

¹Ognuna di queste operazioni o istruzioni è tradotta dal compilatore C in un numero finito (piccolo) costante di passi della macchina fisica sottostante, e può quindi essere considerata a sua volta come un passo elementare

Esempio: Calcolo Massimo di Array

- Dimensione dell'input?

- La lunghezza della porzione di array in cui cercare il massimo

- Quindi:

- La dimensione dell'input è n
- Il tempo di calcolo $T(n)$ (espresso in numero di passi elementari) è
 - Nel caso migliore (il massimo è in posizione 0)
 - $T_b(n) = 3 + 4*n + 5*n + 2 = 5 + 9*n$ ("b" come "best")
 - Nel caso peggiore (array ordinato in ordine strettamente crescente)
 - $T_w(n) = 3 + 4*n + 9*n + 2 = 5 + 13*n$ ("w" come "worst")
- Lo spazio di calcolo $S(n)$ (espresso in numero di celle di memoria) è
 - In tutti i casi: $S_b(n) = S_w(n) = 2$ (le variabili `max` e `i`)

```
/** @brief Assume: lunghezza(a) >= n >= 1 e restituisce:  
* - il massimo che occorre nella porzione a[0..n-1]. */  
                                N PASSI  
  
int maximum(const int[] a, int n){  
    int max = a[0];              3  
    for (int i = 1 ; i < n; i++) { 4*n  
        if (max < a[i]) max = a[i]; tra 5*n e 9*n  
    }  
    return max;                  2  
}
```

Il numero di passi elementari per operazione dipende dal calcolatore: fa parte della sua documentazione

Esempio: Ricerca Elemento in Array

- Dimensione dell'input?

- La lunghezza della porzione di array in cui cercare l'elemento

- Quindi:

- La dimensione dell'input è **n**
- Il tempo di calcolo **T(n)** (espresso in numero di passi elementari) è
 - Nel caso migliore ($key == a[0]$)
 - $T_b(n) = 4*1 + 4*1 + 1 = 9$
 - Nel caso peggiore (key in ultima posizione o non presente)
 - $T_w(n) = 4*n + 4*n + 1 = 1 + 8*n$
- Lo spazio di calcolo **S(n)** (espresso in numero di celle di memoria) è
 - In tutti i casi: $S_b(n) = S_w(n) = 1$ (la variabile i)

```
/** @brief Assume: lunghezza(a) >= n >= 1 e restituisce:  
* - l'indice della prima occorrenza di key in a[0..n-1]  
* - il valore -1 se key non occorre in a[0..n-1]
```

N PASSI

```
int search(const int[] a, size_t n, int key) {  
    for (size_t i = 0; i <= n-1; i++) {  
        if (key == a[i]) return i;  
    }  
    return -1;  
}
```

4*j

4*j

1

Posso interrompere
il ciclo: j è la
posizione di key o n
se non presente



Andamento Asintotico

Complessità: Fattori Costanti

I fattori costanti sono
asintoticamente irrilevanti

- Consideriamo $T_b(n)$ e $T_w(n)$ per `maximum()`:
 - Nel caso migliore (il massimo è in posizione 0): $T_b(n) = 5 + 9 \cdot n$
 - Nel caso peggiore (array ordinato in ordine strettamente crescente): $T_w(n) = 5 + 13 \cdot n$
- E due array di lunghezza n :
 - $a_{b,n}$: un array in cui il massimo è in posizione 0
 - $a_{w,n}$: un array ordinato in ordine strettamente crescente
- Data una macchina M1:
 - $T^{M1}(a_{b,n}) < T^{M1}(a_{w,n})$
 - L'esecuzione di `maximum` con input $a_{b,n}$ richiederà meno tempo rispetto dell'esecuzione di `maximum` con input $a_{w,n}$
- Questa osservazione suggerisce che:
 - Le complessità $T_b(n)$ e $T_w(n)$ possono essere considerate “equivalenti”
 - A parità di n , i casi peggiori su macchina veloce eseguono più velocemente dei casi migliori su macchina lenta
- Data M2 con velocità doppia rispetto a M1:
 - $T^{M1}(a_{b,n}) > T^{M2}(a_{w,n})$
 - L'esecuzione di `maximum` con input $a_{b,n}$ su M1 richiede più tempo dell'esecuzione di `maximum` con input $a_{w,n}$ su M2

M2 dimezza i passi
(cioè i fattori costanti)



Complessità: Fattori Polinomiali

I fattori polinomiali e più in generale **non costanti** sono asintoticamente **rilevanti**

- Consideriamo $T_b(n)$ e $T_w(n)$ per `search()`:
 - Nel caso migliore (`key == a[0]`): $T_b(n) = 9$
 - Nel caso peggiore (`key` in ultima posizione o non presente): $T_w(n) = 1 + 8*n$
 - $a_{b,n}$: un array in cui `key == a[0]`
 - $a_{w,n}$: un array in cui `key` è in ultima posizione o non presente
- Data una macchina M1:
 - $T^{M1}(a_{b,n}) < T^{M1}(a_{w,n})$
 - L'esecuzione di `search` con input $a_{b,n}$ richiederà meno tempo rispetto all'esecuzione di `search` con input $a_{w,n}$
- Questa osservazione suggerisce che:
 - Le complessità $T_b(n) = 9$ e $T_w(n) = 1 + 8*n$ **non possono essere considerate "equivalenti"**
 - Date una macchina veloce e una macchina lenta, a partire da una certa dimensione dell'input n' (che dipende dal rapporto tra le velocità delle due macchine), i casi peggiori sulla macchina veloce, eseguono più lentamente dei casi migliori (della stessa dimensione) sulla macchina lenta
- Data M_m , $m > 1$ volte più veloce di M1 possiamo:
 - Trovare n' tale che:
$$T^{M1}(a_{b,n}) = 9 = (1 + 8*n')/m = T^{Mm}(a_{w,n'})$$
 - E osservare che l'esecuzione di `search` su input $a_{b,n}$ con $n > n'$ su M1 richiede meno tempo di `search` con input $a_{w,n}$ su M_m



Crescita Lineare

- Sia $y = c * x$, con c costante (cioè y proporzionale a x). Allora:
 - se x si raddoppia anche y si raddoppia, se x si triplica anche y si triplica, ...
 - In generale: se x si moltiplica per m anche y si moltiplica per m
- Ovvero, data la funzione $f: \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+$,
 $y = c * x$ (con c costante)
- Se per $x = x_0$ si ha $y = f(x_0) = c * x_0 = y_0$ allora:
 - per $x = 2 * x_0$ si ha $y = f(2 * x_0) = c * 2 * x_0 = 2 * y_0$
 - per $x = 3 * x_0$ si ha $y = f(3 * x_0) = c * 3 * x_0 = 3 * y_0$
 - ...
 - per $x = m$ si ha $y = f(m * x_0) = c * m * x_0 = m * y_0$
- Inoltre, data la funzione $f: y = c * x + d$ (con c e d costanti):
 - Le osservazioni precedenti continuano approssimativamente a valere
 - E l'approssimazione è sempre minore al crescere di x
 - Ovvero: il contributo di d diventa sempre più trascurabile
 - Ad es: per $y = 3 * x + 5$ e $x_1 = 2000$, $x_2 = 1000$:

$$\frac{y_1}{y_2} = \frac{3 * 2000 + 5}{3 * 1000 + 5} = 1.998 \cong 2 = \frac{2000}{1000} = \frac{x_1}{x_2}$$

Crescita Quadratica

- Sia $y = c * x^2$, con c costante. Allora:
 - Se x si raddoppia, allora y si quadruplica, se x si triplica, allora y si moltiplica per 9, ...
 - In generale: se x si moltiplica per m allora y si moltiplica per m^2
- Ovvero, data la funzione $f: \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+$,
 $y = c * x^2$ (con c costante):
- Se per $x = x_0$ si ha $y = f(x_0) = c * x_0^2 = y_0$ allora:
 - per $x = 2 * x_0$ si ha $y = f(2 * x_0) = c * (2 * x_0)^2 = c * 2^2 * x_0^2 = 4 * y_0$
 - per $x = 3 * x_0$ si ha $y = f(3 * x_0) = c * (3 * x_0)^2 = c * 3^2 * x_0^2 = 9 * y_0$
 - ...
 - per $x = m * x_0$ si ha $y = f(m * x_0) = c * (m * x_0)^2 = c * m^2 * x_0^2 = m^2 * y_0$
- Inoltre, data la funzione $f: y = c * x^2 + b * x + a$ (con c , b e a costanti):
 - Le osservazioni precedenti continuano approssimativamente a valere
 - E l'approssimazione è sempre minore al crescere di x
 - Ovvero: i contributi di $b * x$ e a diventano sempre più trascurabili
 - Ad es: per $y = 3 * x^2 + 5 * x + 9$ e $x_1 = 200$, $x_2 = 100$:

$$\frac{y_1}{y_2} = \frac{3 * 40000 + 5 * 200 + 9}{3 * 10000 + 5 * 100 + 9} = 3.966 \cong 4 = \frac{40000}{10000} = \frac{x_1^2}{x_2^2}$$

Nota Importante

- Sia $f: y = c*x^2 + b*x + a$, con costanti b e a molto molto grandi. Allora:
 - La proprietà “*il contributo del termine noto e del termine di primo grado diventa sempre più trascurabile*” diventa osservabile solo per valori molto grandi di x .
- Quindi, dati P1 e P2 che calcolano con algoritmi diversi uno stesso risultato con complessità:
 - $T^{P1}_w(n) = b*n + a$, con b e a molto molto grandi
 - $T^{P2}_w(n) = c*n^2 + d$, con c e d piccole (rispetto a b e a)
- Potrebbe essere che tutti gli input di interesse pratico hanno n per cui
$$T^{P1}_w(n) > T^{P2}_w(n)$$
 - Ovvero P2 con complessità asintoticamente maggiore si comporta nei casi di interesse pratico meglio di P1 con complessità asintoticamente minore
- Oppure potrebbe essere il caso che
 - Fino ad una certa dimensione n_0 di input conviene usare P2
 - Anche se ha complessità asintoticamente maggiore
 - Solo per input di dimensione $> n_0$ conviene usare P1
 - Che ha complessità asintoticamente minore
 - Inoltre, magari arrivati ad una dimensione
 - Usare P1 potrebbe essere **fattibile**
 - Mentre usare P2 potrebbe diventare **irrealizzabile**



Crescita Logaritmica

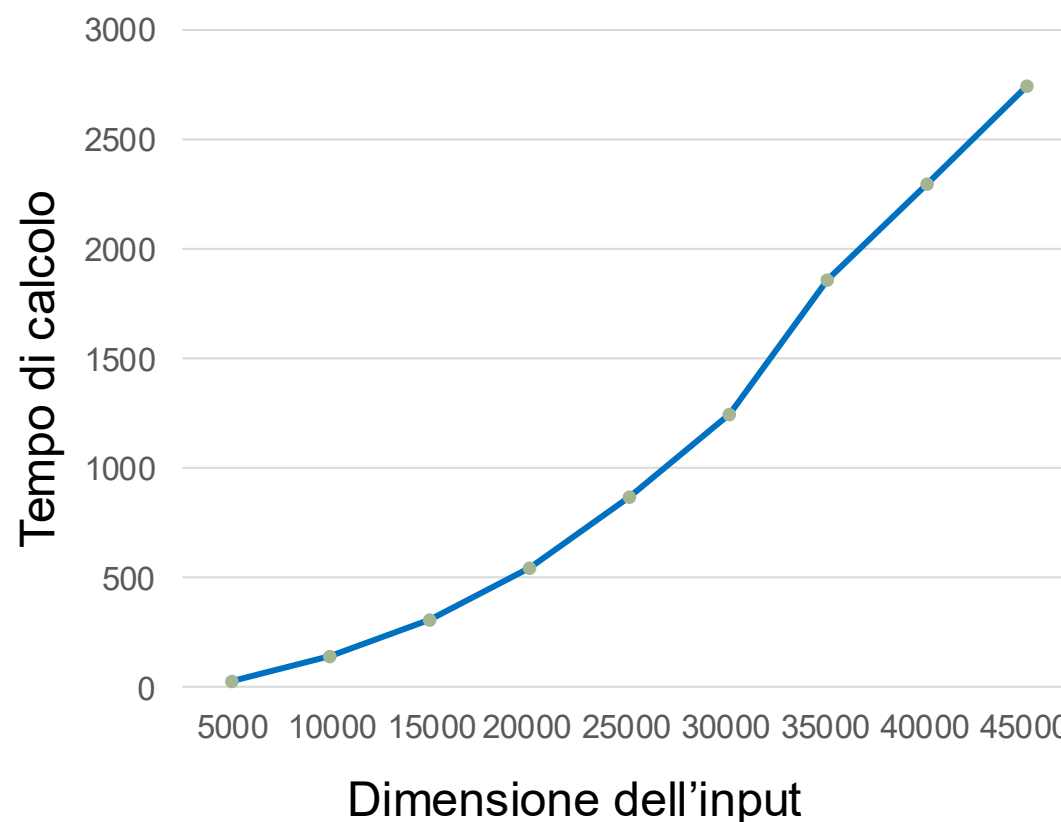


- Due proprietà dei logaritmi:
 - **Prodotto:** $\log_b(x * y) = \log_b(x) + \log_b(y)$ Dimostrazione: $x * y = b^{\log_b(x)} * b^{\log_b(y)} = b^{\log_b(x) + \log_b(y)}$
 - **Cambio di base:** $\log_b(x) = \log_b(a) * \log_a(x)$ Dimostrazione: $\log_b(x) = \log_b(a^{\log_a(x)}) = \log_b(a) * \log_a(x)$
- Sia $y = c * \log_b(x)$, con c e b costanti. Allora:
 - Se x si raddoppia allora y si incrementa di $c * \log_b(2)$, se x si triplica allora y si incrementa di $c * \log_b(3)$, ...
 - In generale: se x si moltiplica per m allora y si incrementa di $c * \log_b(m)$
- Ovvero, data la funzione $f: \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+$, $y = c * \log_b(x)$ (con c e b costanti)
- Se per $x = x_0$ si ha $y = f(x_0) = c * \log_b(x_0) = y_0$ allora:
 - per $x = 2 * x_0$ si ha $y = f(2 * x_0) = c * \log_b(2 * x_0) = c * \log_b(2) + c * \log_b(x_0) = y_0 + c * \log_b(2)$
 - per $x = 3 * x_0$ si ha $y = f(3 * x_0) = c * \log_b(3 * x_0) = c * \log_b(3) + c * \log_b(x_0) = y_0 + c * \log_b(3)$
 - ...
 - per $x = m * x_0$ si ha $y = f(m * x_0) = c * \log_b(m * x_0) = c * \log_b(m) + c * \log_b(x_0) = y_0 + c * \log_b(m)$

Esercizio

- Supponiamo di aver misurato i tempi di esecuzione di una funzione (che implementa un dato algoritmo) per diverse dimensioni dell'input, ottenendo i seguenti risultati
 - Che cosa si può dire dell'andamento asintotico del tempo di calcolo?

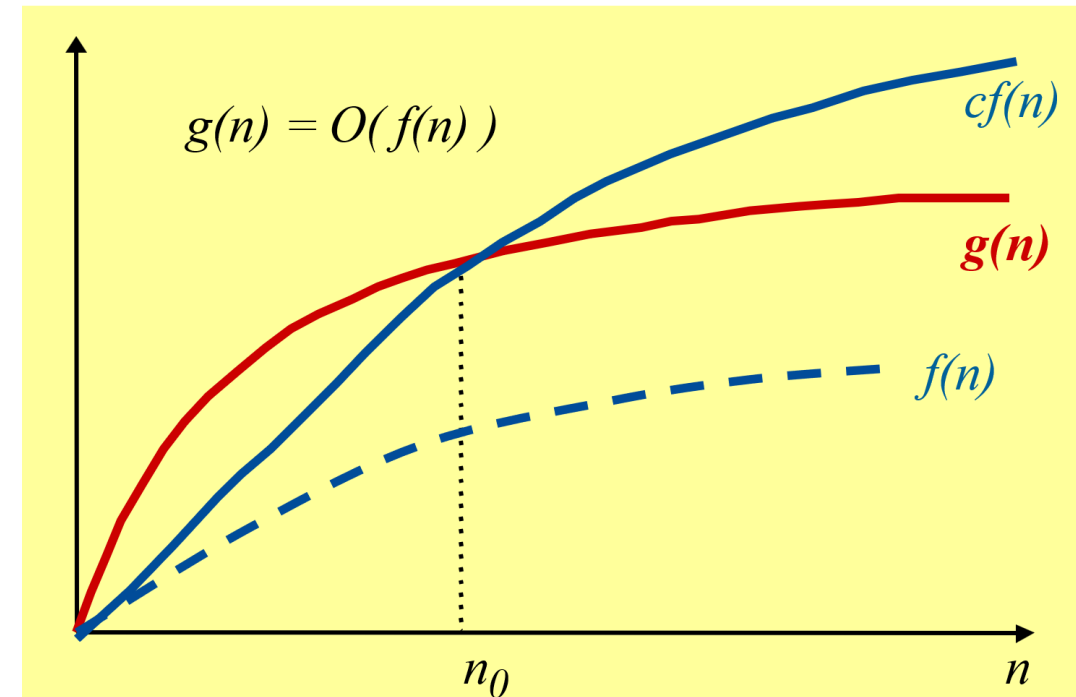
Dimensione dell'input	Tempo di calcolo
5000	31
10000	140
15000	313
20000	547
25000	875
30000	1250
35000	1859
40000	2297
45000	2750



Le Notazioni O e Θ

Andamento Asintotico: O (“O” grande)

- **Informalmente:** “ $g(n) \in O(f(n))$ ” significa
 - “g cresce al più come un multiplo di f”
- **Detto meglio:** “ $g(n) \in O(f(n))$ ” significa
 - “esiste un numero $c > 0$, per cui da un certo numero $n_0 \geq 0$, il valore di f è $\leq$ di c volte il valore di g”
 - La figura a destra illustra un esempio
- **Scritto bene:**



Definizione: l'insieme $O(f(n))$

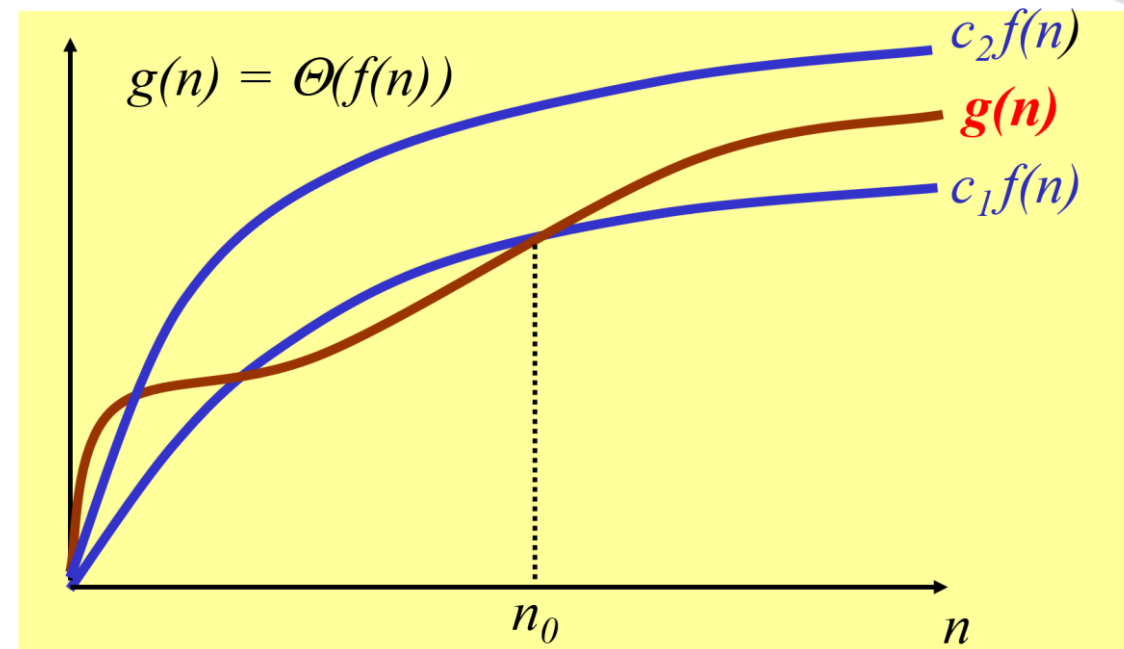
Data una funzione $f: \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+$, definiamo:

$$O(f(n)) = \{h: \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+ \mid \exists c > 0, \exists n_0 \geq 0, \text{ tali che } \forall n \geq n_0 \text{ si ha } h(n) \leq c * f(n)\}$$

Con un abuso di notazione consolidato, in letteratura si scrive “ $g(n) = O(f(n))$ ” o “ $g(n)$ è $O(f(n))$ ” per indicare “ $g(n) \in O(f(n))$ ”

Andamento Asintotico: Θ (“Theta” grande)

- **Informalmente:** “ $g(n) \in \Theta(f(n))$ ” significa
 - “g cresce come f”
- **Detto meglio:** “ $g(n) \in \Theta(f(n))$ ” significa
 - “ci sono due numeri $c_2 \geq c_1 > 0$, per cui da un certo numero $n_0 \geq 0$, il valore di f è compresa tra c_1 volte il valore di g e c_2 volte il valore di g”
- La figura a destra illustra un esempio
- **Scritto bene:**



Definizione: l'insieme $\Theta(f(n))$

Data una funzione $f: \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+$, definiamo:

$$\Theta(f(n)) = \{h: \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+ \mid \exists c_1, c_2 > 0, \exists n_0 \geq 0, \text{ tali che } \forall n \geq n_0 \text{ si ha } c_1 * f(n) \leq h(n) \leq c_2 * f(n)\}$$

Con un abuso di notazione consolidato, in letteratura si scrive “ $g(n) = \Theta(f(n))$ ” o “ $g(n)$ è $\Theta(f(n))$ ” per indicare “ $g(n) \in \Theta(f(n))$ ”

Esempio: Ricerca Elemento in Array Ordinato

- Dimensione dell'input?

- La lunghezza della porzione di array ordinate in cui cercare l'elemento

```
/** @brief Assume: n >= 1, a[0..n-1] ordinato in ordine
crescente. Restituisce: -1'indice di key nella porzione
a[0..n-1] o il valore -1 se key non occorre in a[0..n-1] */
int binSearch(const int[] a, size_t n, int key) {
    size_t low = 0;
    size_t high = n-1;
    while (low <= high) {
        size_t middle = (low + high) / 2;
        if (key < a[middle]) {high = middle-1;}
        else if (key > a[middle]) {low = middle+1;}
        else {return middle;}
    }
    return -1;
}
```

N PASSI (W)

$\Theta(1)$

$\Theta(1)$

$\Theta(\log_2(n))$

$\Theta(\log_2(n))$

$\Theta(\log_2(n))$

$\Theta(\log_2(n))$

$\Theta(1)$

$\Theta(1)$

Totale: $\Theta(\log_2(n))$

- Quindi:

- La dimensione dell'input è n
- Il tempo di calcolo $T(n)$ (espresso in numero di passi elementari) è
 - Nel caso migliore ($key == a[(n-1)/2]$) $T_b(n) = \Theta(1)$
 - Nel caso peggiore (key non presente, o solo in prima/ultima posizione, o ...) $T_w(n) = \Theta(\log_2(n))$
- Lo spazio di calcolo (espresso in numero di celle di memoria) è
 - $S_b(n) = S_w(n) \in \Theta(1)$ (le 3 variabili di tipo `size_t`)

Complessità Computazionale: Caso Peggior e Caso Migliore

Definizioni

- Sia **A** un algoritmo (ad esempio espresso in linguaggio C)
- Sia **I** un possibile input per un'esecuzione di **A**
- Sia **t(I)** il tempo di esecuzione dell'algoritmo **A** con input **I**
- Sia **s(I)** lo spazio di memoria necessario per l'esecuzione dell'algoritmo **A**
 - **Senza considerare** lo spazio occupato dall'input **I**
- Allora la **complessità temporale** di **A**:
 - Nel **caso peggiore** è la funzione $T_w(n) = \max \{ t(I) \mid I \text{ ha dimensione } n \}$
 - Nel **caso migliore** è la funzione $T_b(n) = \min \{ t(I) \mid I \text{ ha dimensione } n \}$
- Le definizioni per la **complessità spaziale** sono perfettamente analoghe
 - Basta sostituire **T** con **S**, e sostituire **t** con **s**

Esempio: Calcolo Massimo di Array

- Dimensione dell'input?

- La lunghezza della porzione di array in cui cercare il massimo

- Quindi:

- La dimensione dell'input è **n**
- Il tempo di calcolo **$T(n)$** (espresso in numero di passi elementari) è
 - **$T_b(n), T_w(n) \in \Theta(n)$**
- Lo spazio di calcolo **$S(n)$** (espresso in numero di celle di memoria) è
 - In tutti i casi: **$S_b(n) = S_w(n) \in \Theta(1)$** (le variabili `max` e `i`)

```
/** @brief Assume: lunghezza(a) >= n >= 1 e restituisce:  
* - il massimo che occorre nella porzione a[0..n-1]. */  
                                     N PASSI  
  
int maximum(const int[] a, int n){  
    int max = a[0];                   $\Theta(1)$   
    for (int i = 1 ; i < n; i++) {    $\Theta(n)$   
        if (max < a[i]) max = a[i];   $\Theta(n)$   
    }  
    return max;                       $\Theta(1)$   
}                                     TOTALE  $\Theta(n)$ 
```

Esempio: Ricerca Elemento in Array

- Dimensione dell'input?

- La lunghezza della porzione di array in cui cercare l'elemento

- Quindi:

- La dimensione dell'input è **n**
- Il tempo di calcolo **$T(n)$** (espresso in numero di passi elementari) è
 - Nel caso migliore ($key == a[0]$) **$T_b(n) \in \Theta(1)$**
 - Nel caso peggiore (key in ultima posizione o non presente) **$T_w(n) \in \Theta(n)$**
- Lo spazio di calcolo **$S(n)$** (espresso in numero di celle di memoria) è
 - In tutti i casi: **$S_b(n) = S_w(n) \in \Theta(1)$** (la variabile i)

```
/** @brief Assume: lunghezza(a) >= n >= 1 e restituisce:  
* - l'indice della prima occorrenza di key in a[0..n-1]  
* - il valore -1 se key non occorre in a[0..n-1]
```

N PASSI

```
int search(const int[] a, size_t n, int key) {  
    for (size_t i = 0; i <= n-1; i++) {  
        if (key == a[i]) return i;  
    }  
    return -1;  
}.  
  
TOTAL  $\Theta(n)$ 
```

Esempio: Ricerca Elemento in Array Ordinato

- Dimensione dell'input?

- La lunghezza della porzione di array ordinate in cui cercare l'elemento

```
/** @brief Assume: n >= 1, a[0..n-1] ordinato in ordine
crescente. Restituisce: -1'indice di key nella porzione
a[0..n-1] o il valore -1 se key non occorre in a[0..n-1] */
int binSearch(const int[] a, size_t n, int key) {
    size_t low = 0;
    size_t high = n-1;
    while (low <= high) {
        size_t middle = (low + high) / 2;
        if (key < a[middle]) {high = middle-1;}
        else if (key > a[middle]) {low = middle+1;}
        else {return middle;}
    }
    return -1;
}
```

N PASSI (W)

$\Theta(1)$

$\Theta(1)$

$\Theta(\log_2(n))$

$\Theta(\log_2(n))$

$\Theta(\log_2(n))$

$\Theta(\log_2(n))$

$\Theta(1)$

$\Theta(1)$

Totale: $\Theta(\log_2(n))$

- Quindi:

- La dimensione dell'input è n
- Il tempo di calcolo $T(n)$ (espresso in numero di passi elementari) è
 - Nel caso migliore ($key == a[(n-1)/2]$) $T_b(n) = \Theta(1)$
 - Nel caso peggiore (key non presente, o solo in prima/ultima posizione, o ...) $T_w(n) = \Theta(\log_2(n))$
- Lo spazio di calcolo (espresso in numero di celle di memoria) è
 - $S_b(n) = S_w(n) \in \Theta(1)$ (le 3 variabili di tipo `size_t`)

Confronto: `search` vs `binSearch`



- La dimensione dell'input è n , la lunghezza della porzione di array in cui cercare l'elemento.
- Per `search` (anche su array non ordinato)
 - Lo spazio di calcolo (in # celle di memoria) è
 - $S_b(n) = S_w(n) \in \Theta(1)$ (una variabile)
 - Il tempo di calcolo (in # passi elementari) è
 - Nel caso migliore ($key == a[0]$)
 - $T_b(n) \in \Theta(1)$
 - Nel caso peggiore (key in ultima posizione o non presente)
 - $T_w(n) \in \Theta(n)$
- I casi migliori di `search` sono casi peggiori di `binSearch`
 - Per array molto piccoli (fino a 5 elementi?), `search` potrebbe essere più conveniente di `binSearch`
- Per `binSearch` (solo su array ordinato)
 - Lo spazio di calcolo (in # celle di memoria) è
 - $S_b(n) = S_w(n) \in \Theta(1)$ (tre variabili)
 - Il tempo di calcolo (in # passi elementari) è
 - Nel caso migliore ($key == a[(n-1)/2]$)
 - $T_b(n) = \Theta(1)$
 - Nel caso peggiore (key non presente, o solo in prima/ultima posizione, o ...)
 - $T_w(n) = \Theta(\log_2(n))$

Esercizio



- Dato:
 - La funzione:
 - `int smartSearch(const int[] a, size_t n, int key)`
 - che utilizza la ricerca sequenziale per restituire
 - L'indice dell'ultima occorrenza di `key` nella porzione di array ordinata `a[0..n-1]`
 - Il valore -1 se `key` non occorre in `a`
 - fermandosi non appena incontra un elemento `> key`?
- Scrivete:
 - Una colonna per `smartSearch` analoga alle colonne per `search` e `binSearch` presentate nella slide precedente

Commenti Finali

- In queste slide ci focalizziamo sull'andamento asintotico del numero di operazioni elementari
 - Dal punto di vista asintotico, tutte le operazioni elementari sono uguali
- Tuttavia, sperimentalmente, in certi algoritmi, alcune operazioni elementari hanno un tempo di esecuzione più alto
 - In tal caso si parla di “operazione dominante”
 - Esempio: operazioni di confronto tra chiavi particolarmente complesse in algoritmi di ordinamento e di ricerca
- In tali casi, si preferisce contare (magari, prima di passare alla notazione asintotica, in modo preciso) il numero di operazioni dominanti
 - Naturalmente, bisogna verificare che l'andamento asintotico del numero di operazioni dominanti sia lo stesso del numero di operazioni elementari (in generale)!

